# HOSPITAL PATIENT DATA ANALYSIS

## A SUMMER INTERNSHIP REPORT

*Submitted by*

**Venmani V– 810023205057**




**DEPARTMENT OF INFORMATION TECHNOLOGY**

**ANNA UNIVERSITY,**

**TIRUCHIRAPPALLI – 620 024.**

**UNIVERSITY COLLEGE OF ENGINEERING (BIT CAMPUS),**

**ANNA UNIVERSITY, TIRUCHIRAPPALLI – 620 024**

**DEPARTMENT OF INFORMATION TECHNOLOGY**

# BONAFIDE CERTIFICATE

This is to certify that this Summer Internship project report **"Hospital Patient Data Analysis"** is the bonafide work of **Venmani V (810023205057)** who has satisfactorily completed the course of **IT3711 – SUMMER INTERNSHIP** for **SEVENTH** semester, **B.TECH INFORMATION TECHNOLOGY** during the academic year **2025-2026**.

**Faculty Incharge** **Head of the Department**

Submitted for the practical examination held on _____

# ABSTRACT

The **Hospital Patient Data Analysis** project is a data analysis solution that turns raw hospital patient records into clear, usable insights. Hospitals generate large volumes of data on admissions, diagnoses, treatments, billing, length of stay and discharge outcomes. Held in separate files and registers, this data is difficult to use for decisions. This project brings it together into one clean dataset and analyses it systematically.

The work is implemented using **Python** (Pandas, NumPy, Matplotlib, Seaborn), **SQL (MySQL)** for storage and querying, and **Jupyter Notebook** for development. Results are presented through charts and an interactive dashboard. The pipeline covers data collection, cleaning, feature engineering, exploratory data analysis (EDA), visualisation and insight reporting.

The main objectives are to **identify admission and disease trends, analyse length of stay and readmission patterns, study department-wise patient load and treatment cost, and support hospital management in planning resources**. By replacing manual spreadsheet-based reporting with automated analysis, the project improves accuracy, saves time and supports data-driven decision-making.

Overall, the project demonstrates the practical use of the complete data analysis life cycle on healthcare data and shows how analytics can improve hospital operations and patient care.

# TABLE OF CONTENTS


| CHAPTER | TITLE | PAGE NO |
|---|---|---|
| 1 | **INTRODUCTION**<br>1.1 Objectives of the Internship, 1.2 Motivation | 3 |
| 2 | **COMPANY PROFILE** | 5 |
| 3 | **DAILY WORK REPORT** | 7 |
| 4 | **BLOCK DIAGRAM** | 9 |
| 5 | **HOSPITAL PATIENT DATA ANALYSIS SYSTEM**<br>5.1 Existing System, 5.2 Proposed System, 5.3 Capabilities, 5.4 System Requirements | 9 |
| 6 | **PROPOSED METHOD**<br>6.1 Data Collection to 6.8 Database Design | 12 |
| 7 | **ALGORITHM**<br>7.1 Data Loading to 7.7 Insight Generation | 15 |
| 8 | **CODE IMPLEMENTATION**<br>8.1 Data Loading to 8.10 Saving Cleaned Data | 16 |
| 9 | **SAMPLE OUTPUT**<br>9.1 Data Loading to 9.11 Dashboard | 21 |
| 10 | **ANALYSIS**<br>10.1 Data Quality to 10.6 Challenges and Solutions | 29 |
| 11 | **CONCLUSION**<br>Learning Outcomes | 32 |
| 12 | **FUTURE WORK** | 32 |
| 13 | **REFERENCES** | 33 |

# 1. INTRODUCTION

**Hospital Patient Data Analysis** is a data analysis project that studies patient records to find patterns in admissions, diagnoses, treatments, costs and outcomes. Modern hospitals record every patient visit digitally, but the data is often scattered across files, contains missing or inconsistent values, and is rarely analysed in a structured way. Management therefore depends on manual reports that are slow and error-prone.

The primary purpose of this project is to build a clean, reliable analysis workflow that converts raw patient data into meaningful insights. The workflow collects data from hospital records, cleans and validates it, performs exploratory analysis, and presents the findings through charts and dashboards that doctors and administrators can understand easily.

The project is developed using **Python** (Pandas, NumPy, Matplotlib, Seaborn) for processing and visualisation, **MySQL** for storing and querying the patient data, **Jupyter Notebook** for development, and a dashboard for interactive reporting. Statistical measures such as mean, median, correlation and group-wise aggregation are used to summarise the data.

The system follows a **three-layer architecture** consisting of the Data Layer (raw data and database), the Processing Layer (cleaning, transformation and analysis) and the Presentation Layer (charts and dashboards). Key analysis modules include Data Cleaning, Patient Demographics Analysis, Admission and Disease Trend Analysis, Length of Stay and Readmission Analysis, Billing and Treatment Cost Analysis, and Dashboard Reporting.

The project gives practical knowledge of the data analysis life cycle, SQL querying, data visualisation and healthcare analytics. It can be extended with predictive models for readmission risk, bed-demand forecasting and real-time monitoring in future versions.

## 1.1 OBJECTIVES OF THE INTERNSHIP

The main objective is to analyse hospital patient data using data analysis techniques and to present insights that help in better planning and patient care. The objectives of the project are:

- To collect and organise hospital patient data into a structured dataset using **Python and MySQL**.
- To **clean the data** by handling missing values, duplicates, inconsistent formats and outliers.
- To study **patient demographics** such as age group, gender and location.
- To identify **admission trends, common diagnoses** and seasonal patterns.
- To analyse **length of stay, readmission rates** and discharge outcomes.
- To analyse **department-wise patient load, treatment cost** and billing.
- To create clear **visualisations and an interactive dashboard** for decision-makers.
- To derive insights that help **reduce waiting time, improve bed utilisation** and plan resources.
- To demonstrate the practical application of **Python, SQL, statistics and data visualisation**.

## 1.2 MOTIVATION

The motivation behind this project is to make hospital data useful. Many hospitals still depend on manual reports and spreadsheets, which causes delays, inaccurate figures and missed patterns. Analysing the data properly helps management see which diseases are increasing, which departments are overloaded and where costs can be controlled.

Another motivation is to reduce manual reporting effort and improve decision quality. Automated analysis saves time and gives consistent results every time new data arrives.

The project also gave hands-on experience with a complete data analysis workflow, from raw data to a finished dashboard, using tools widely used in industry.

## 1.3 SCOPE OF THE PROJECT

The scope of this project covers the analysis of hospital patient records for one calendar year. It includes data collection, data cleaning, feature engineering, exploratory analysis, visualisation and dashboard reporting. The analysis focuses on admissions, diagnoses, length of stay, readmissions and billing. Clinical decisions about individual patients are

outside the scope of this project; the results are meant only to support management planning.

### 1.4 ORGANISATION OF THE REPORT

The report is organised into thirteen chapters. Chapter 1 introduces the project and its objectives. Chapter 2 describes the company. Chapter 3 gives the daily work report. Chapters 4 to 7 explain the block diagram, the existing and proposed systems, the method and the algorithms. Chapter 8 presents the code, and Chapter 9 shows the sample outputs. Chapter 10 gives the analysis and test results, Chapter 11 the conclusion, Chapter 12 the future work and Chapter 13 the references.

## 2. COMPANY PROFILE




### About the Company

**Innoknowvex Pvt Ltd** is a technology and training company based in Bengaluru, Karnataka. It provides software, data and technology-based solutions and offers industrial training and internship programmes that connect academic learning with real industry practice.

During the internship, students work on real-time projects under the guidance of an industry expert. Exposure includes data collection, data cleaning, analysis, visualisation, reporting and documentation. Internships follow a structured project-based approach with regular review by the team leader.

As part of my internship at Innoknowvex Pvt Ltd, I worked on **Hospital Patient Data Analysis** under the guidance of **Mr. D. Kumaresan, Team Leader**. The project involved collecting and cleaning patient data, running SQL queries, performing exploratory analysis in Python, building visualisations and presenting insights through a dashboard.

The internship improved my skills in Python, SQL, statistics, data visualisation and problem solving, and helped me understand how analytics supports decisions in the healthcare domain.

## Company Details

| Item | Details |
|---|---|
| Company Name | Innoknowvex Pvt Ltd |
| Address | Sb01-no 581/a, Rupena Agrahara, Sector IV, HSR Layout, Bengaluru, Karnataka – 560102, India |
| Industry | Information Technology, Training and Data Solutions |
| Industry Expert (Joint Supervisor) | Mr. D. Kumaresan, Team Leader |
| Internship Domain | Data Analysis |
| Project Completed | Hospital Patient Data Analysis |
| Internship Period | 12 June 2026 – 12 July 2026 |

## Technologies Used

| Area | Tools |
|---|---|
| Programming Languages | Python, SQL |
| Libraries | Pandas, NumPy, Matplotlib, Seaborn |
| Database | MySQL |
| Visualisation | Matplotlib, Seaborn, Dashboard tools |
| Development Tools | Jupyter Notebook, VS Code |
| Version Control | Git and GitHub |

## 3. DAILY WORK REPORT

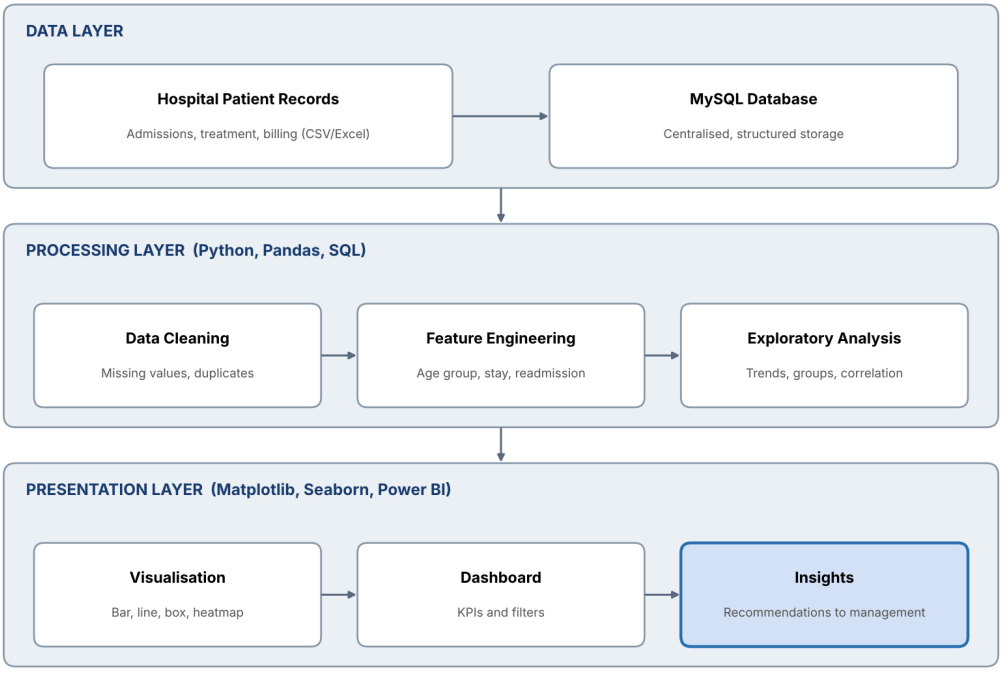
| Day | Work Done |
|---|---|
| Day 1 | Attended orientation; briefed on internship guidelines, evaluation criteria and reporting format. Discussed and finalised the project topic "Hospital Patient Data Analysis" with the industry expert. |
| Day 2 | Gathered requirements by discussing with the mentor. Listed the stakeholders (hospital administrator, doctor, data analyst) and the key questions the analysis must answer. |
| Day 3 | Reviewed existing hospital analytics practices and reports. Documented the drawbacks of manual, spreadsheet-based patient reporting. |
| Day 4 | Finalised the scope and objectives of the project. Prepared the list of hardware and software requirements. |
| Day 5 | Set up the environment: Python 3, Anaconda, Jupyter Notebook, MySQL and Git. Verified the setup with a sample notebook. |
| Day 6 | Studied Pandas and NumPy fundamentals: DataFrames, indexing, filtering, grouping and merging. |
| Day 7 | Designed the overall architecture using Data, Processing and Presentation layers. Prepared the block diagram. |
| Day 8 | Identified the dataset and designed the schema with entities Patient, Admission, Diagnosis, Treatment, Department and Billing. |
| Day 9 | Prepared the Entity-Relationship (ER) diagram showing relationships between the entities. |
| Day 10 | Created the MySQL database and tables, and loaded the raw patient data. |
| Day 11 | Explored the data: shape, data types, summary statistics and a data dictionary of all columns. |
| Day 12 | Analysed missing values column by column and decided the treatment strategy for each. |
| Day 13 | Handled missing values and removed duplicate records. |
| Day 14 | Standardised formats: dates, text case, categorical labels and units. |
| Day 15 | Detected outliers in age, length of stay and bill amount using the IQR method. |
| Day 16 | Created new features: age group, length of stay, admission month and readmission flag. |
| Day 17 | Wrote SQL queries using joins, GROUP BY and aggregate functions to validate the cleaned data. |
| Day 18 | Performed patient demographic analysis by age group and gender. |

| Day | Work Done |
|---|---|
| Day 19 | Analysed monthly and seasonal admission trends. |
| Day 20 | Analysed the most common diagnoses and disease distribution. |
| Day 21 | Analysed department-wise patient load and bed utilisation. |
| Day 22 | Analysed length of stay by department and diagnosis. |
| Day 23 | Analysed readmission rates and the factors linked to them. |
| Day 24 | Analysed billing and treatment cost across departments and diagnoses. |
| Day 25 | Performed correlation analysis between age, stay, cost and readmission, and plotted a heatmap. |
| Day 26 | Created visualisations with Matplotlib and Seaborn: bar, line, pie, box and histogram charts. |
| Day 27 | Designed the dashboard layout and selected the key performance indicators (KPIs). |
| Day 28 | Built the dashboard with KPI cards and charts for department, month and diagnosis. |
| Day 29 | Prepared the key insights and recommendations for hospital management. |
| Day 30 | Validated results, fixed issues, took screenshots for documentation and compiled the final internship report. |

## Weekly Summary

| Week | Days | Work Completed |
|---|---|---|
| Week 1 | Day 1 – 7 | Orientation, requirement gathering, scope, environment setup, Pandas and NumPy study, architecture design |
| Week 2 | Day 8 – 14 | Database schema and ER diagram, MySQL setup, data exploration, missing value handling, duplicate removal, format standardisation |
| Week 3 | Day 15 – 21 | Outlier detection, feature engineering, SQL validation, demographic, trend, diagnosis and department analysis |
| Week 4 | Day 22 – 28 | Length of stay, readmission and cost analysis, correlation, visualisations, dashboard design and development |
| Final days | Day 29 – 30 | Insights and recommendations, validation, documentation and final report |

# 4. BLOCK DIAGRAM




*Block diagram of the Hospital Patient Data Analysis system*

Patient records are loaded into the MySQL database, then cleaned, enriched with new features and analysed in the processing layer. The presentation layer turns the results into charts, a dashboard and recommendations for hospital management.

# 5. HOSPITAL PATIENT DATA ANALYSIS SYSTEM

## 5.1 Existing System

Most hospitals still manage patient information through registers, separate spreadsheets and basic reports. Admission, treatment, billing and discharge details are often stored in different places. Reports are prepared manually, so they are slow to produce and depend on individual staff. This existing approach suffers from several limitations:

- **Scattered data** – Patient, treatment and billing records are kept in separate files, which makes combined analysis difficult.
- **Poor data quality** – Missing values, duplicates and inconsistent formats lead to wrong figures.
- **No trend analysis** – Management cannot easily see admission trends, seasonal diseases or recurring problems.
- **Manual reporting** – Reports take hours or days and are prone to human error.

- **Limited visualisation** – Tables of numbers are hard for doctors and administrators to interpret.
- **Weak resource planning** – Without clear patient-load and length-of-stay data, beds and staff are not planned efficiently.
- **Hidden readmission patterns** – Patients who return within a short time are not identified or studied.
- **Uncontrolled cost** – Department-wise and diagnosis-wise cost is not analysed, so cost control is difficult.

These drawbacks show the need for a centralised, automated and visual analysis system.

## 5.2 Proposed System

The proposed system is a **data analysis workflow** that combines patient data into one database, cleans it, analyses it and presents the results visually. It replaces manual reporting with repeatable analysis that can be rerun whenever new data arrives. The system uses Python and SQL for processing, MySQL for storage, and charts and dashboards for reporting. Key aspects of the proposed system include:

- **Centralised data** – Patient, admission, treatment and billing data are stored in a single MySQL database.
- **Automated cleaning** – Missing values, duplicates, wrong formats and outliers are handled with Python scripts.
- **Exploratory analysis** – Demographics, admissions, diagnoses, stay length, readmissions and costs are summarised using statistics.
- **Trend detection** – Monthly and seasonal patterns are identified from admission dates.
- **Visual reporting** – Charts and a dashboard present results to non-technical users.
- **Decision support** – Insights and recommendations help management plan beds, staff and budget.

By automating these steps, the system reduces manual effort, improves accuracy and gives faster, better-informed decisions.

## 5.3 Capabilities

- **Data Management** – Loads, stores and organises patient data in a MySQL database.

- **Data Cleaning** – Handles missing values, duplicates, inconsistent formats and outliers.
- **Demographic Analysis** – Studies patients by age group and gender.
- **Admission Trend Analysis** – Finds monthly and seasonal admission patterns.
- **Diagnosis Analysis** – Identifies the most common diseases and their share.
- **Length of Stay Analysis** – Measures average stay by department and diagnosis.
- **Readmission Analysis** – Finds readmission rates and the factors related to them.
- **Cost Analysis** – Analyses billing and treatment cost by department.
- **Visualisation and Dashboard** – Presents charts and KPIs for quick understanding.
- **Reporting** – Produces insights and recommendations for hospital management.

## 5.4 System Requirements

The project was developed and tested on a standard personal computer. The hardware and software requirements are given below.

| Hardware | Requirement |
|---|---|
| Processor | Intel Core i3 or higher |
| RAM | 8 GB or more |
| Storage | 256 GB SSD or higher |
| Display | 1366 x 768 resolution or higher |

| Software | Requirement |
|---|---|
| Operating System | Windows 10 / 11 |
| Programming Language | Python 3.x |
| Libraries | Pandas, NumPy, Matplotlib, Seaborn |
| Database | MySQL 8.0 |
| Development Environment | Jupyter Notebook, VS Code |
| Version Control | Git and GitHub |
| Browser | Google Chrome (for viewing the dashboard) |

## 5.5 Advantages of the Proposed System

- Saves time by replacing manual report preparation with automated analysis.

- Improves accuracy through systematic data cleaning and validation.
- Gives management a clear view of admissions, costs and outcomes.
- Helps in planning beds, staff and budget before peak periods.
- Can be reused whenever new patient data is available.
- Uses free and widely available tools, so the cost of development is low.

### 5.6 Limitations

- The analysis depends on the quality and completeness of the input data.
- The system describes past data and does not predict future admissions.
- Clinical details such as test results and treatment plans are not included.
- Data privacy rules must be followed strictly when real patient data is used.

## 6. PROPOSED METHOD

### 6.1 Data Collection

- Collects hospital patient data covering patient details, admission and discharge dates, department, diagnosis, treatment, length of stay and billing.
- Loads the data from CSV/Excel files into a MySQL database.
- Keeps patient identity fields anonymised so that privacy is protected.
- Maintains a data dictionary describing every column.

### 6.2 Data Cleaning and Preprocessing

- Identifies and treats missing values using suitable methods (median for numeric, "Unknown" for categorical).
- Removes duplicate patient and admission records.
- Converts dates to a standard format and standardises text labels.
- Detects outliers using the IQR method and caps them.

### 6.3 Feature Engineering

- Creates age groups: Child, Adult and Senior.
- Calculates length of stay as discharge date minus admission date.
- Extracts admission month for trend analysis.
- Creates a readmission flag for patients who return within 30 days.

### 6.4 Exploratory Data Analysis

- Summarises data using mean, median, standard deviation and counts.
- Performs group-wise analysis by department, diagnosis, gender and age group.
- Studies relationships between age, length of stay, cost and readmission using correlation.
- Uses SQL queries with joins and aggregation to cross-check Python results.

### 6.5 Data Visualisation

- Creates bar charts for department load and diagnosis counts.
- Creates line charts for monthly admission trends.
- Creates box plots for length of stay.
- Creates a correlation heatmap for numeric features.

### 6.6 Dashboard Development

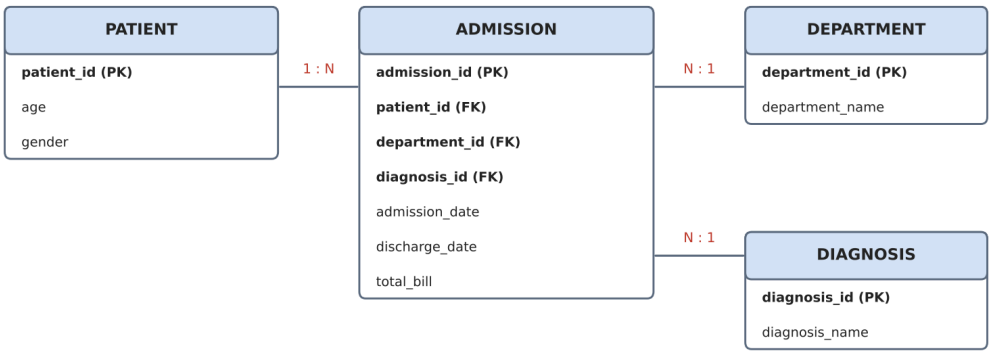
- Designs a dashboard with KPIs such as total admissions, average stay, readmission rate and average bill.
- Arranges charts so that management can read the main findings at a glance.

### 6.7 Insight Reporting

- Converts the results into clear findings and recommendations.
- Highlights overloaded departments, high-cost departments and seasonal peaks.
- Documents the method, results and screenshots in the final report.

## 6.8 Database Design

The patient data is stored in a normalised MySQL database. The Admission table is the central table and is linked to the Patient, Department and Diagnosis tables through foreign keys. This avoids repeated data and keeps the records consistent.




*ER Diagram of the Hospital Patient Database*

**Data Dictionary**

| Column | Data Type | Description |
|---|---|---|
| patient_id | Integer | Unique identifier of the patient (anonymised) |
| age | Integer | Age of the patient in years |
| gender | Text | Gender of the patient (Male / Female) |
| department | Text | Department in which the patient was admitted |
| diagnosis | Text | Primary diagnosis of the patient |
| admission_date | Date | Date of admission to the hospital |
| discharge_date | Date | Date of discharge from the hospital |
| total_bill | Decimal | Total treatment bill in INR |
| length_of_stay | Integer | Derived: number of days between admission and discharge |
| age_group | Text | Derived: Child, Adult or Senior |
| admission_month | Text | Derived: year and month of admission |
| readmitted | Integer | Derived: 1 if readmitted within 30 days, otherwise 0 |

# 7. ALGORITHM

## 7.1 Data Loading – Algorithm

1. Read the patient data file (CSV/Excel) into a Pandas DataFrame.
2. Check the number of rows, columns and data types.
3. Validate that all required columns are present.
4. Create the MySQL tables if they do not exist.
5. Insert the records into the database.
6. Confirm that the database row count matches the file.

## 7.2 Data Cleaning – Algorithm

1. Count missing values in every column.
2. Fill numeric gaps with the median and categorical gaps with "Unknown".
3. Remove duplicate records.
4. Convert date columns to a standard date format.
5. Standardise text labels (case, spelling, spaces).
6. Detect outliers using Q1, Q3 and the IQR.
7. Treat the outliers and save the cleaned dataset.

## 7.3 Feature Engineering – Algorithm

1. Take age from the dataset and assign each patient to an age group.
2. Compute length of stay = discharge date − admission date.
3. Extract the admission month.
4. Mark a patient as readmitted if the next admission occurs within 30 days of discharge.
5. Add the new columns to the dataset.

## 7.4 Exploratory Analysis – Algorithm

1. Generate summary statistics for numeric columns.
2. Count patients by gender, age group, department and diagnosis.
3. Group by department and compute average stay, average bill and readmission rate.
4. Group by month and count admissions.
5. Calculate the correlation matrix of numeric features.
6. Store the results for visualisation.

### 7.5 Visualisation – Algorithm

1. Select the metric and the chart type that suits it.
2. Prepare the grouped data for the chart.
3. Plot the chart with title, axis labels and legend.
4. Check the chart for clarity and correctness.
5. Save the chart image for the report.

### 7.6 Dashboard Reporting – Algorithm

1. Load the cleaned dataset.
2. Create KPI cards for admissions, average stay, readmission rate and average bill.
3. Add charts for trends, departments, diagnoses and gender.
4. Check that every chart matches the analysis results.
5. Save the dashboard image.

### 7.7 Insight Generation – Algorithm

1. Review the results of all analyses.
2. Identify the highest and lowest values and unusual patterns.
3. Link each pattern to a likely cause or action.
4. Write recommendations for management.
5. Include them in the final report with supporting charts.

## 8. CODE IMPLEMENTATION

### 8.1 Data Loading Module

Loads the patient dataset and checks its structure.

```
import pandas as pd
import numpy as np

df = pd.read_csv("hospital_patients.csv")

print(df.shape)
print(df.dtypes)
print(df.head())
print(df.describe(include="all"))
```

### 8.2 Data Cleaning Module

Handles missing values, duplicates, date formats and outliers.

```
# Missing values
print(df.isnull().sum())
df["age"] = df["age"].fillna(df["age"].median())
df["total_bill"] = df["total_bill"].fillna(df["total_bill"].median())
df["diagnosis"] = df["diagnosis"].fillna("Unknown")

# Duplicates
df = df.drop_duplicates()

# Standard formats
df["admission_date"] = pd.to_datetime(df["admission_date"])
df["discharge_date"] = pd.to_datetime(df["discharge_date"])
df["gender"] = df["gender"].str.strip().str.title()
df["department"] = df["department"].str.strip().str.title()

# Outliers using IQR (capping)
q1 = df["total_bill"].quantile(0.25)
q3 = df["total_bill"].quantile(0.75)
iqr = q3 - q1
lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr
df["total_bill"] = df["total_bill"].clip(lower, upper)
```

## 8.3 Feature Engineering Module

Creates new columns used in the analysis.

```
df["length_of_stay"] = (df["discharge_date"] -
df["admission_date"]).dt.days

df["age_group"] = pd.cut(df["age"], bins=[0, 17, 59, 120],
                         labels=["Child", "Adult", "Senior"])

df["admission_month"] =
df["admission_date"].dt.to_period("M").astype(str)

# Readmission within 30 days
df = df.sort_values(["patient_id", "admission_date"])
df["prev_discharge"] =
df.groupby("patient_id")["discharge_date"].shift(1)
df["gap_days"] = (df["admission_date"] - df["prev_discharge"]).dt.days
df["readmitted"] = (df["gap_days"] <= 30).astype(int)
```

## 8.4 Exploratory Analysis Module

Summarises patients by group and computes key measures.

```
dept_load = df["department"].value_counts()

dept_summary = df.groupby("department").agg(
    patients=("patient_id", "count"),
    avg_stay=("length_of_stay", "mean"),
    avg_bill=("total_bill", "mean"),
    readmission_rate=("readmitted", "mean")
).round(2)

top_diagnosis = df["diagnosis"].value_counts().head(10)
monthly = df.groupby("admission_month").size()
```

```
corr = df[["age", "length_of_stay", "total_bill", "readmitted"]].corr()
```

### 8.5 Visualisation Module

Plots the main charts used in the report.

```
import matplotlib.pyplot as plt
import seaborn as sns

dept_load.plot(kind="barh", title="Patients by Department")
plt.xlabel("Number of Patients")
plt.tight_layout()
plt.show()

monthly.plot(kind="line", marker="o", title="Monthly Admissions")
plt.ylabel("Admissions")
plt.show()

sns.boxplot(data=df, x="department", y="length_of_stay")
plt.xticks(rotation=35)
plt.show()

sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```

### 8.6 Database Creation Module

Creates the MySQL database and the tables used to store the patient data.

```
CREATE DATABASE hospital_db;
USE hospital_db;

CREATE TABLE patients (
    patient_id INT PRIMARY KEY,
    age INT,
    gender VARCHAR(10)
);

CREATE TABLE departments (
    department_id INT PRIMARY KEY AUTO_INCREMENT,
    department_name VARCHAR(50) UNIQUE
);

CREATE TABLE diagnoses (
    diagnosis_id INT PRIMARY KEY AUTO_INCREMENT,
    diagnosis_name VARCHAR(60) UNIQUE
);

CREATE TABLE admissions (
    admission_id INT PRIMARY KEY AUTO_INCREMENT,
    patient_id INT,
    department_id INT,
    diagnosis_id INT,
    admission_date DATE,
    discharge_date DATE,
    total_bill DECIMAL(12,2),
```

```
    FOREIGN KEY (patient_id) REFERENCES patients(patient_id),
    FOREIGN KEY (department_id) REFERENCES departments(department_id),
    FOREIGN KEY (diagnosis_id) REFERENCES diagnoses(diagnosis_id)
);
```

## 8.7 SQL Query Module

Cross-checks the Python results directly in MySQL.

```
-- Patients and average stay by department
SELECT department,
       COUNT(*) AS patients,
       AVG(DATEDIFF(discharge_date, admission_date)) AS avg_stay
FROM admissions
GROUP BY department
ORDER BY patients DESC;

-- Total billing by diagnosis
SELECT diagnosis, SUM(total_bill) AS total_billing
FROM admissions
GROUP BY diagnosis
ORDER BY total_billing DESC
LIMIT 10;
```

## 8.8 Additional Visualisation Module

Draws the demographic charts, histograms and the department-month heatmap.

```
sns.countplot(data=df, x="age_group", hue="gender",
              order=["Child", "Adult", "Senior"])
plt.title("Patients by Age Group and Gender")
plt.show()

sns.histplot(df["age"], bins=20)
plt.title("Age Distribution of Patients")
plt.show()

sns.histplot(df["total_bill"], bins=25)
plt.title("Distribution of Total Bill")
plt.show()

pivot =
df.assign(month=df["admission_date"].dt.strftime("%b")).pivot_table(
    index="department", columns="month",
    values="patient_id", aggfunc="count")
sns.heatmap(pivot, annot=True, fmt="g", cmap="YlOrRd")
plt.title("Admissions by Department and Month")
plt.show()

readm_age = df.groupby("age_group")["readmitted"].mean() * 100
readm_age.plot(kind="bar", title="Readmission Rate by Age Group (%)")
plt.show()
```

## 8.9 Dashboard Module

Combines the KPI cards and the main charts on a single page.

```
fig = plt.figure(figsize=(14, 8.4))
gs = fig.add_gridspec(3, 4, height_ratios=[0.5, 1.6, 1.6])

kpis = [("Total Admissions", f"{len(df):,}"),
        ("Avg Length of Stay", f"{df['length_of_stay'].mean():.1f}
days"),
        ("Readmission Rate", f"{df['readmitted'].mean() * 100:.1f}%"),
        ("Avg Bill", f"INR {df['total_bill'].mean():,.0f}")]

for i, (title, value) in enumerate(kpis):
    ax = fig.add_subplot(gs[0, i])
    ax.axis("off")
    ax.text(0.5, 0.65, value, ha="center", fontsize=18,
fontweight="bold")
    ax.text(0.5, 0.2, title, ha="center", fontsize=10)

ax = fig.add_subplot(gs[1, :2])
monthly.plot(marker="o", ax=ax, title="Monthly Admissions")

ax = fig.add_subplot(gs[1, 2:])
dept_load.sort_values().plot(kind="barh", ax=ax, title="Patients by
Department")

ax = fig.add_subplot(gs[2, 0])
df["gender"].value_counts().plot(kind="pie", ax=ax, autopct="%1.0f%%")

ax = fig.add_subplot(gs[2, 1:3])
top_diagnosis.head(6).sort_values().plot(kind="barh", ax=ax, title="Top
Diagnoses")

plt.savefig("dashboard.png", dpi=140)
```

## 8.10 Saving Cleaned Data

Stores the cleaned dataset for further reporting.

```
df.to_csv("hospital_patients_clean.csv", index=False)
```

# 9. SAMPLE OUTPUT

The outputs below were produced by running the code in Chapter 8 on the hospital patient dataset.

## 9.1 Data Loading Output

Displays the shape and first records of the raw dataset after loading.

```
In [1]: df.shape, df.head()

Shape: (2025, 8)

   patient_id   age  gender        department   diagnosis admission_date discharge_date  total_bill
0         159  24.0    Male         Neurology    Epilepsy     2025-06-20     2025-06-24     91994.0
1         785  62.0    male       Orthopedics   Back Pain     2025-02-08     2025-02-19     82459.0
2         325  20.0  Female         Emergency  Chest Pain     2025-08-19     2025-08-20     31530.0
3        1130  24.0  Female        Obstetrics   C-Section     2025-12-04     2025-12-07     63975.0
4         895  32.0    Male  General Medicine    Diabetes     2025-10-02     2025-10-04     17810.0
```

*Raw Dataset Shape and First Five Records*

## 9.2 Summary Statistics

Shows the count, mean, standard deviation, minimum, quartiles and maximum of the numeric columns.

```
In [4]: df[['age','length_of_stay','total_bill']].describe()

              age  length_of_stay  total_bill
count  2005.00         2005.00     2005.00
mean     38.99            4.71    66598.50
std      20.92            2.85    50290.65
min       1.00            1.00    10182.00
25%      24.00            2.00    24559.00
50%      38.00            4.00    47791.00
75%      54.00            6.00    99729.00
max      95.00           15.00   212484.00
```

*Summary Statistics of Numeric Columns*

## 9.3 Data Cleaning Output

Shows the missing values before and after cleaning, along with the number of duplicates removed and outliers capped.

```
In [2]: missing values before and after cleaning

                before  after
patient_id           0      0
age                 61      0
gender               0      0
department           0      0
diagnosis           31      0
admission_date       0      0
discharge_date       0      0
total_bill          47      0

Duplicates removed: 20
Outliers capped (IQR): 39
```

*Missing Values Before and After Cleaning*

## 9.4 Department Summary

Summarises patient count, average stay, average bill and readmission rate for each department.

```
In [3]: dept_summary

                patients  avg_stay   avg_bill  readmission_rate
department
Cardiology           259      6.04  119696.10              0.10
Emergency            336      2.38   41469.12              0.13
General Medicine     452      4.25   25043.06              0.12
Neurology            162      6.30  109235.43              0.09
Obstetrics           188      3.05   57495.49              0.13
Oncology             123      8.06  163378.75              0.15
Orthopedics          257      7.32   94853.09              0.12
Pediatrics           228      3.07   18848.68              0.09
```

*Department-wise Summary Table*

## 9.5 SQL Query Output

Shows the results of the SQL queries run in MySQL to cross-check the Python results.

```
MySQL query output


mysql> SELECT department, COUNT(*) AS patients, AVG(DATEDIFF(discharge_date, admission_date)) AS avg_stay
    -> FROM admissions GROUP BY department ORDER BY patients DESC;

department          patients  avg_stay
General Medicine       452.0    4.2456
Emergency              336.0    2.3780
Cardiology             259.0    6.0425
Orthopedics            257.0    7.3230
Pediatrics             228.0    3.0658
Obstetrics             188.0    3.0479
Neurology              162.0    6.2963
Oncology               123.0    8.0569

8 rows in set
```

*SQL Output: Patients and Average Stay by Department*

```
MySQL query output


mysql> SELECT diagnosis, SUM(total_bill) AS total_billing FROM admissions
    -> GROUP BY diagnosis ORDER BY total_billing DESC LIMIT 5;

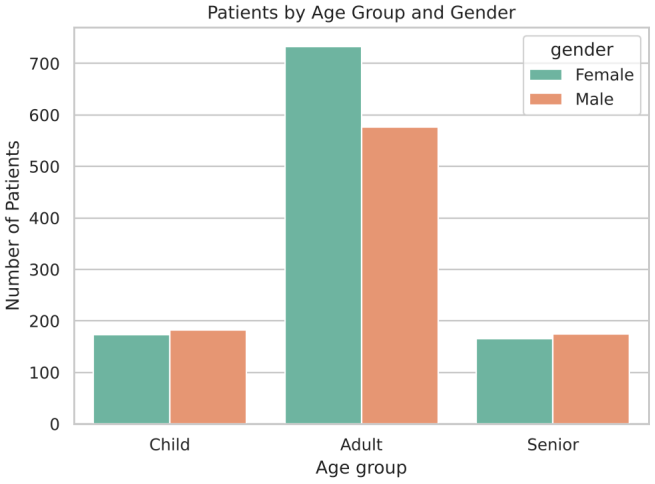
diagnosis            total_billing
Arrhythmia              11,440,782
Heart Failure            9,797,870
Angina                   9,762,637
Fracture                 9,099,611
Back Pain                8,102,569

5 rows in set
```

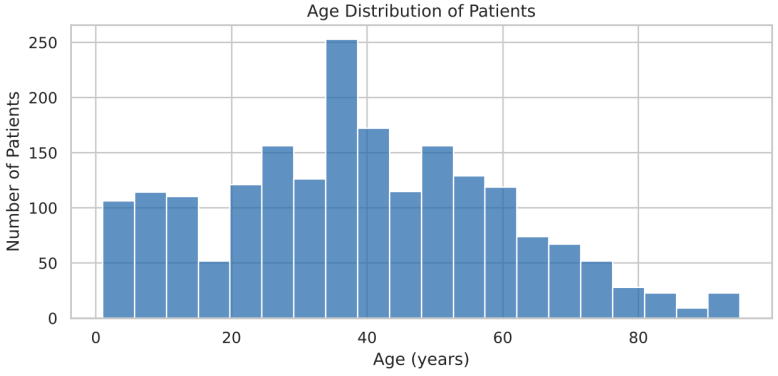
*SQL Output: Total Billing by Diagnosis*

## 9.6 Patient Demographics

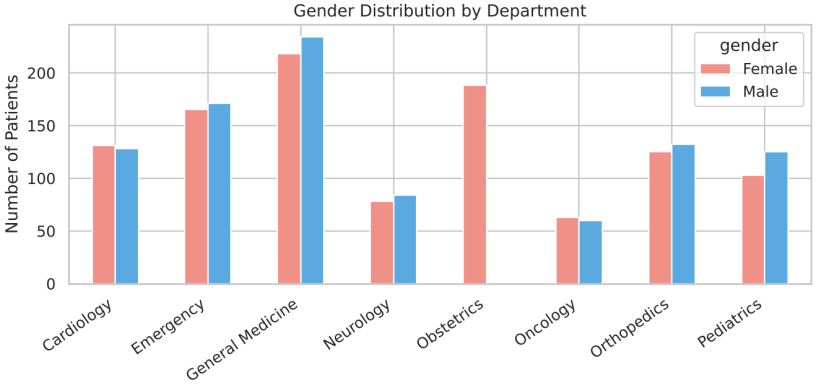
Shows the distribution of patients by age group, gender and age.




*Patients by Age Group and Gender*



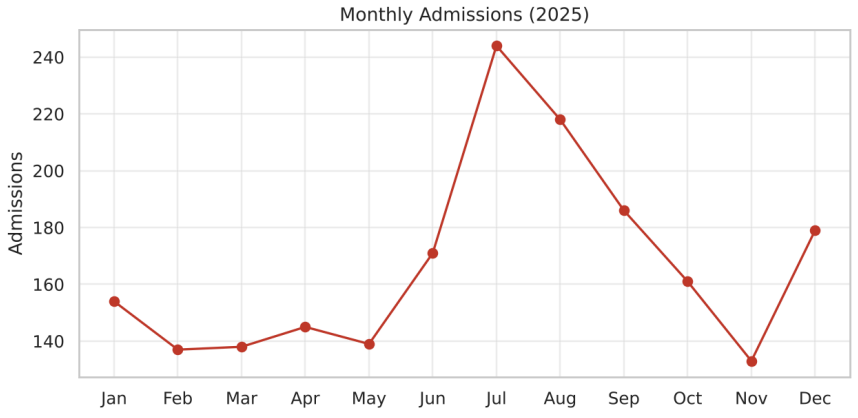

*Age Distribution of Patients*



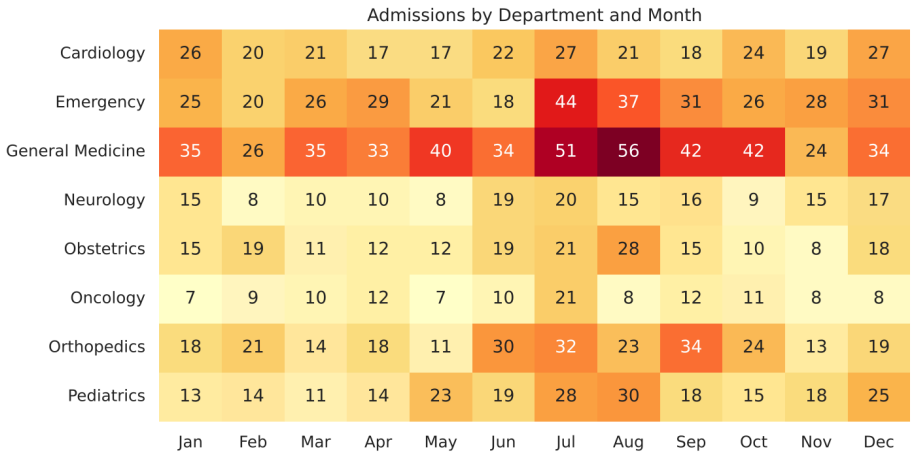

*Gender Distribution by Department*

## 9.7 Admission Trend

A line chart of monthly admissions that reveals seasonal peaks and low periods, and a heatmap of admissions by department and month.



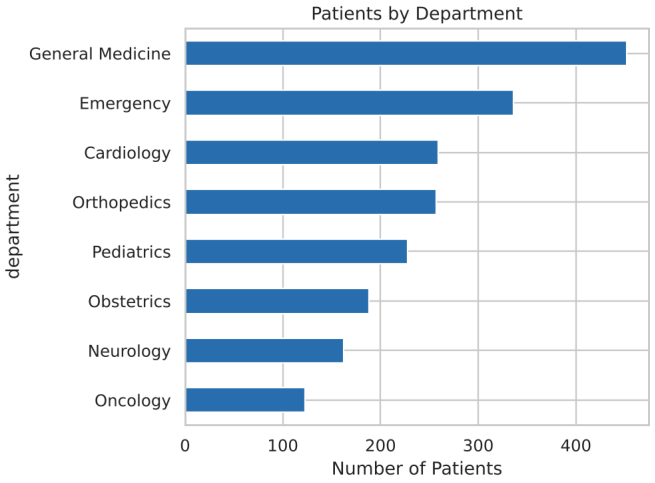

*Monthly Admission Trend*



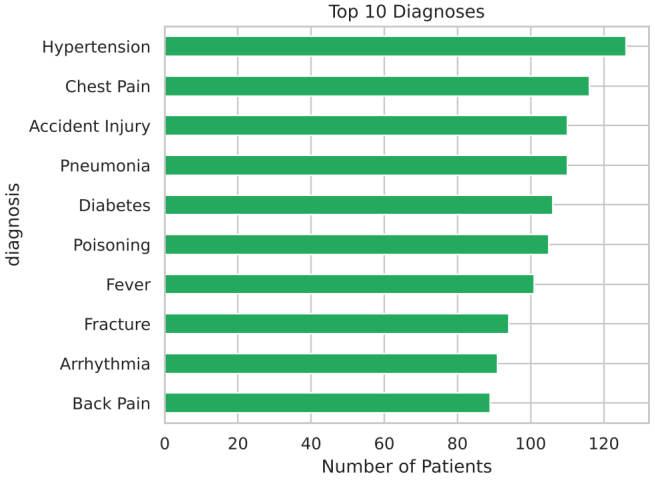

*Admissions by Department and Month*

## 9.8 Department Load and Diagnoses

Bar charts comparing the number of patients in each department and the most common diagnoses.



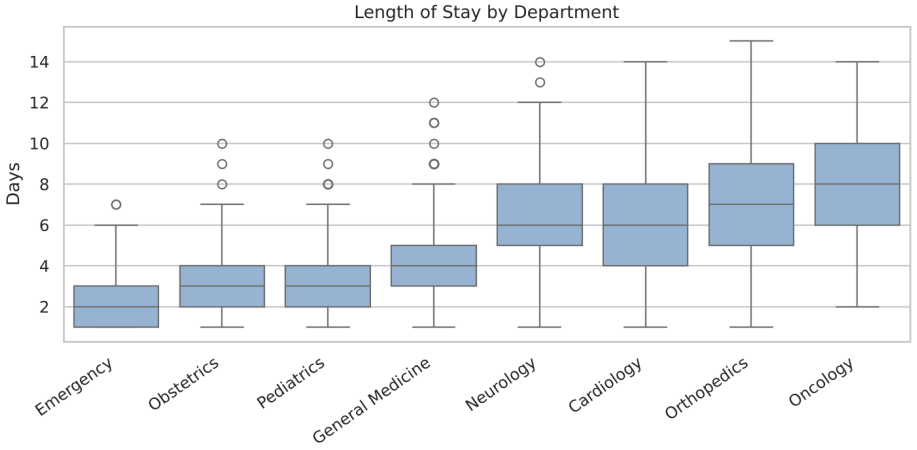

*Patients by Department*



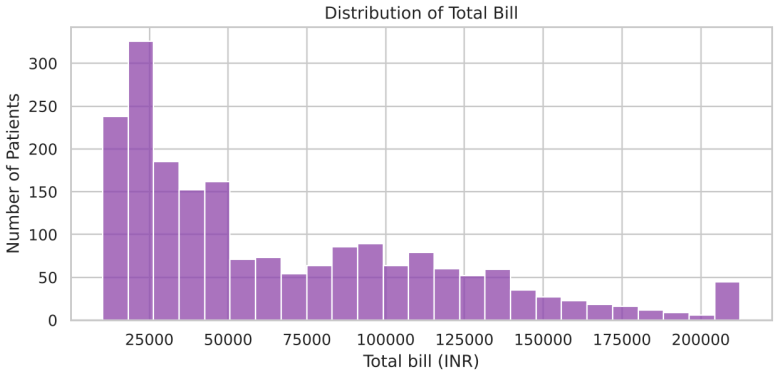

*Top 10 Diagnoses*

## 9.9 Length of Stay and Billing

Box plots of length of stay in each department and the distribution of the total bill.



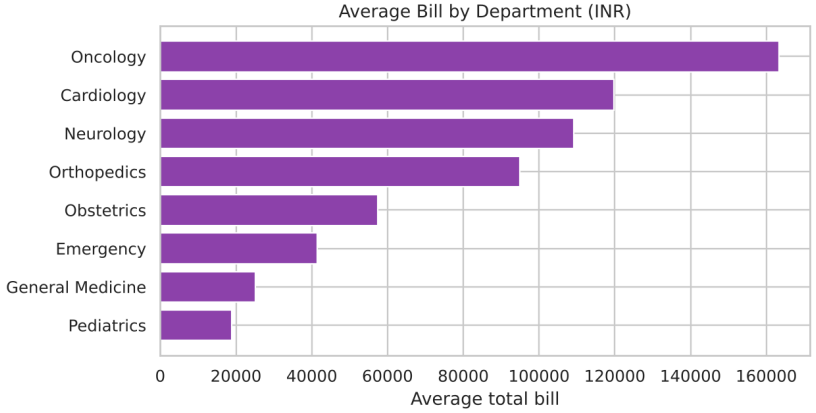

*Length of Stay by Department*



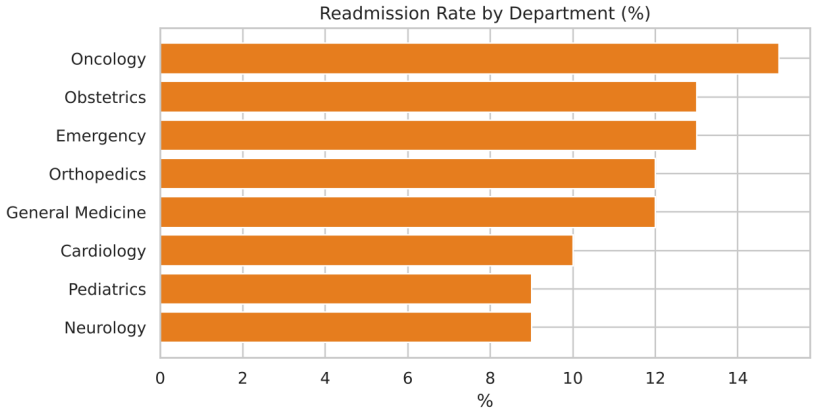

*Distribution of Total Bill*

## 9.10 Cost, Readmission and Correlation

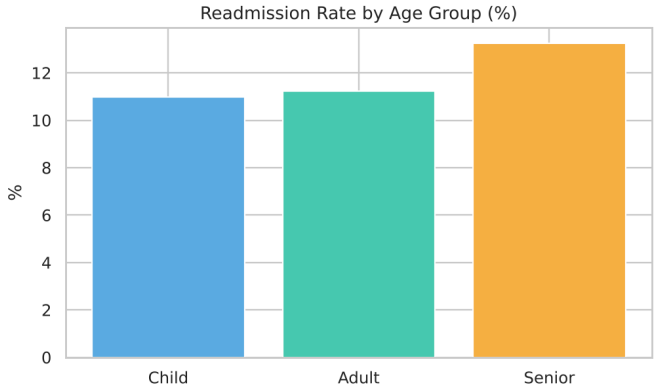
Average bill and readmission rate by department, readmission by age group, and the correlation between the numeric features.



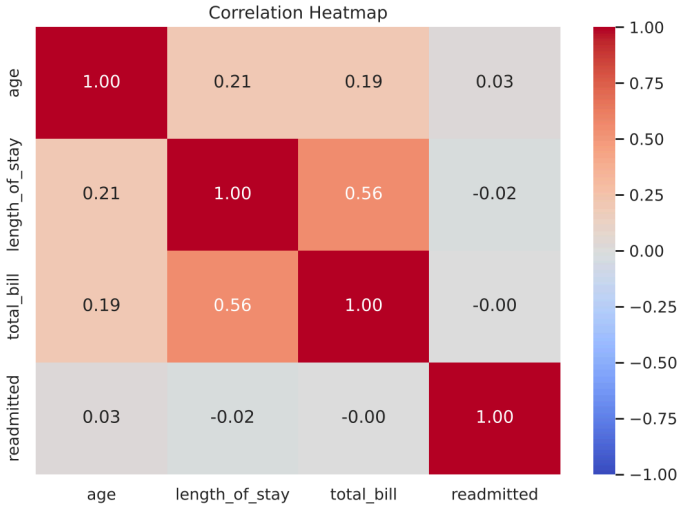

*Average Bill by Department*




*Readmission Rate by Department*



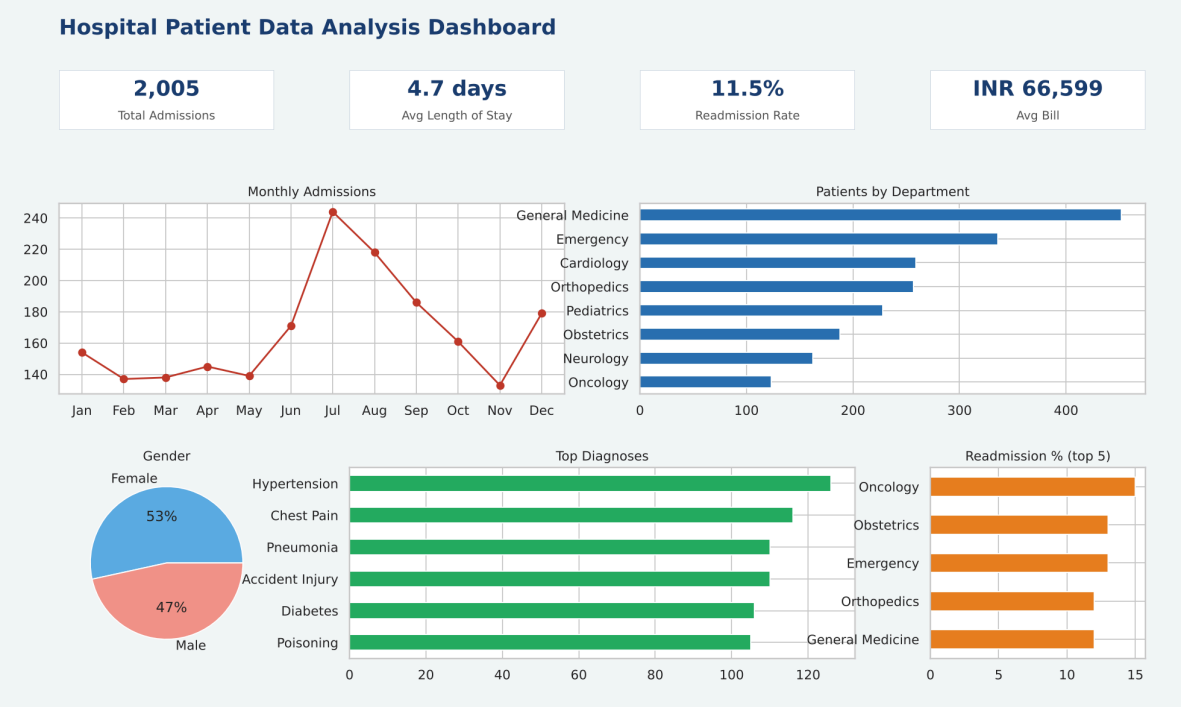

*Readmission Rate by Age Group*




*Correlation Heatmap*

### 9.11 Dashboard

The final dashboard combines the KPI cards and the main charts on a single page for hospital management.




*Hospital Patient Data Analysis Dashboard*

## 10. ANALYSIS

The analysis was carried out stage by stage to check both the quality of the data and the correctness of the results.

### 10.1 Data Quality Analysis

The raw dataset had 2,025 records and 8 columns. Cleaning removed 20 duplicate records, filled missing values in age, diagnosis and total bill, standardised dates and labels, and capped 39 bill outliers using the IQR method. After cleaning, 2,005 records remained and no missing values were left.

Key figures such as patients per department, average length of stay and total billing were calculated in Python and cross-checked with SQL queries on MySQL. Both methods gave matching results, which confirmed the correctness of the analysis.

## 10.2 Test Cases

Each module was tested with normal and edge-case inputs. The results are given below.

| No. | Module | Test Input | Expected Result | Result |
|---|---|---|---|---|
| 1 | Data loading | Valid CSV file | All rows and columns loaded | Pass |
| 2 | Missing values | Records with empty age, diagnosis and bill | No missing values remain | Pass |
| 3 | Duplicates | Repeated patient records | Duplicate rows removed | Pass |
| 4 | Date conversion | Text dates in the file | Converted to date type | Pass |
| 5 | Outlier treatment | Very high bill amounts | Values capped within IQR limits | Pass |
| 6 | Length of stay | Admission and discharge dates | Correct number of days | Pass |
| 7 | Age group | Ages from 1 to 95 | Child, Adult or Senior assigned | Pass |
| 8 | Readmission flag | Patients with repeat visits | Flag 1 only within 30 days | Pass |
| 9 | Group analysis | Department and month grouping | Counts match SQL results | Pass |
| 10 | Charts and dashboard | Cleaned dataset | All charts drawn correctly | Pass |

## 10.3 Key Findings

- The hospital recorded an **average length of stay of 4.7 days**, an **overall readmission rate of 11.5%** and an **average bill of ₹66,599**.
- **General Medicine** had the highest patient load with 452 admissions, followed by Emergency and Cardiology.
- Admissions peaked in **Jul** (244 admissions) and were lowest in **Nov** (133 admissions), showing a clear seasonal pattern.
- **Hypertension** was the most frequent diagnosis with 126 patients.
- **Oncology** had the longest average stay (8.1 days), the highest average bill (₹1,63,379) and the highest readmission rate (15%).

- Length of stay and bill amount are **strongly positively correlated** (r = 0.56), while age and length of stay show a weak positive correlation (r = 0.21).
- Senior patients (60 years and above) form about 17% of admissions.

### 10.4 Recommendations

- Plan additional staff and beds before the peak admission months.
- Give special attention to Oncology, where stay, cost and readmission are all highest.
- Monitor readmission cases closely and improve follow-up after discharge.
- Use the dashboard for regular monthly review by hospital management.

### 10.5 Comparison with the Manual Approach

Manual reporting needs hours of work and is prone to errors. The analysis workflow produces the same reports in minutes, can be rerun when new data arrives, and presents results visually so that management can act on them quickly.

Overall, the analysis confirms that the project meets its objectives of cleaning and organising patient data, finding useful patterns and supporting better planning in the hospital.

### 10.6 Challenges Faced and Solutions

| Challenge | Solution |
|---|---|
| Missing values in age, diagnosis and bill | Filled numeric gaps with the median and text gaps with "Unknown" so that no record was lost |
| Duplicate patient records | Removed exact duplicates using drop_duplicates() after checking the counts |
| Very high bill amounts distorting averages | Detected outliers with the IQR method and capped them within the limits |
| Inconsistent text such as gender in lower case | Standardised labels with strip() and title() before grouping |
| Identifying readmitted patients | Sorted by patient and date, and compared each admission with the previous discharge |
| Making charts easy to understand | Chose simple chart types with clear titles, labels and consistent colours |

# 11. CONCLUSION

The **Hospital Patient Data Analysis** project shows how data analysis can turn raw hospital records into useful decisions. By replacing scattered, manual reporting with a clean database and a repeatable analysis workflow, the project reduces effort, improves accuracy and gives management a clear view of patient activity.

The project covered the full data analysis life cycle: data collection, cleaning, feature engineering, exploratory analysis, visualisation, dashboard development and insight reporting. Tools such as Python, Pandas, SQL, MySQL, Matplotlib and Seaborn were used together.

The internship at Innoknowvex Pvt Ltd gave valuable practical experience in Python programming, SQL, statistics, data visualisation and healthcare analytics. The work can serve as a foundation for more advanced analytics, including prediction and real-time monitoring.

## Learning Outcomes

- Understood the complete **data analysis life cycle** from data collection to reporting.
- Gained hands-on skills in **Python, Pandas and NumPy** for data cleaning and analysis.
- Learned to write **SQL queries** with joins, grouping and aggregate functions in MySQL.
- Learned to choose and create suitable **visualisations** with Matplotlib and Seaborn.
- Understood how to design a **dashboard** that communicates results to non-technical users.
- Gained knowledge of **healthcare data** and the importance of patient data privacy.
- Improved **problem solving, documentation and teamwork** skills in an industry environment.

# 12. FUTURE WORK

- Building a **machine learning model** to predict patient readmission risk.
- **Forecasting** bed demand and admissions using time-series models.
- Predicting **length of stay** to help with bed allocation.
- Connecting the workflow to a **live hospital database** for real-time dashboards.

- Adding **disease outbreak detection** from sudden rises in diagnoses.
- Analysing **patient feedback and satisfaction** data.
- Adding **role-based dashboards** for doctors, nurses and administrators.
- Strengthening **data privacy and security** with anonymisation and access control.
- Deploying the dashboard on a **cloud platform** such as AWS, Azure or GCP.
- Adding **automated email reports** for management.

## 13. REFERENCES


1. Pandas Documentation – Data structures, cleaning and analysis in Python.
2. NumPy Documentation – Numerical computing in Python.
3. Matplotlib Documentation – Plotting and chart customisation.
4. Seaborn Documentation – Statistical data visualisation.
5. MySQL Documentation – Relational database management and SQL.
6. Python Official Documentation – Language features and standard library.
7. Jupyter Documentation – Notebook-based development.
8. GeeksforGeeks – Python, SQL and data analysis concepts.
9. W3Schools – SQL and Python learning resources.